**Computación de Alto Rendimiento (HPC)**

Reto 2 — Monte Carlo para el cálculo de $\pi$

Concurrencia con OpenMP, perfilado y benchmark

# Algoritmos de Monte Carlo para estimar $\pi$:

Método del dartboard y de la aguja de Buffon,
con análisis de paralelización y optimización

José Alejandro Quintero Gahona

Meycol Kleym Quintero Castaño

María Angélica Alvarez Giraldo

Docente: Ramiro Andrés Barrios Valencia

Asignatura: Computación de Alto Rendimiento

10 de noviembre de 2026

# Índice


**1. Introducción y objetivos** **3**

**2. Los dos algoritmos y su modelo matemático** **4**
2.1. Método del dartboard . . . . . 4
2.2. La aguja de Buffon . . . . . 4
2.3. Distribución del error . . . . . 4
2.4. Consecuencia: el dartboard es un estimador más preciso . . . . . 5
2.5. Consecuencia: el dartboard no usa trigonometría . . . . . 5

**3. Implementación serial** **5**
3.1. Estructura del código . . . . . 5
3.2. El generador de números aleatorios . . . . . 6
3.3. Determinismo . . . . . 6
3.4. Las versiones paralelas no son idénticas, y está bien . . . . . 6
3.5. Modo de validación . . . . . 7

**4. Análisis de opciones de paralelización y optimización** **7**
4.1. El generador de números aleatorios: el verdadero cuello de botella . . . . . 7
4.2. El contador de aciertos: la optimización de memoria . . . . . 8
4.3. El hallazgo más interesante: el aliasing . . . . . 8
4.4. La política de reparto del trabajo . . . . . 9
4.5. Resumen de las variantes implementadas . . . . . 9

**5. Diseño experimental** **9**
5.1. Variables . . . . . 9
5.2. Qué se mide y con qué estimador . . . . . 10
5.3. Validación de efectividad . . . . . 10
5.4. Protocolo de comparabilidad entre máquinas . . . . . 10
5.5. Herramientas . . . . . 11

**6. Resultados** **11**
6.1. Validación de efectividad . . . . . 11
6.2. Convergencia hacia $\pi$ . . . . . 11
6.3. Perfilado de CPU: el generador de números aleatorios domina . . . . . 12
6.4. Perfilado de memoria: el conjunto de trabajo cabe en la caché . . . . . 12
6.5. Desempeño: escalamiento de las cuatro variantes . . . . . 14
6.6. Costo por muestra . . . . . 14
6.7. Benchmark entre las dos máquinas . . . . . 15

**7. Análisis de resultados** **16**
7.1. El cuello de botella es el generador de números aleatorios, no la aritmética . 16
7.2. Por qué el dartboard es 5.5 veces más rápido que Buffon . . . . . 17
7.3. Por qué el dartboard también es más preciso . . . . . 18
7.4. El perfilado de memoria: qué cambia y qué no . . . . . 18

7.5. Por qué el dartboard escala peor que Buffon, siendo más rápido . . . . . 18
7.6. El efecto del aliasing: la explicación del problema más importante . . . . . 19
7.7. El estado del generador y el determinismo . . . . . 19
7.8. Qué nos enseña la comparación entre las dos máquinas . . . . . 19
7.9. Una salvedad sobre el estado del sistema . . . . . 20
7.10. Qué se mantiene igual entre máquinas . . . . . 20

**8. Conclusiones** **21**
8.1. Hallazgos . . . . . 21
8.2. Limitaciones . . . . . 22
8.3. Trabajo futuro . . . . . 22

**9. Referencias** **23**

# 1. Introducción y objetivos

## Qué se hizo

Este documento presenta el Reto 2 de la materia de Computación de Alto Rendimiento (HPC). La consigna pide cuatro cosas: analizar los algoritmos de Monte Carlo de la **aguja de Buffon** y del **dartboard** para calcular $\pi$, implementar las versiones **seriales** de ambos, analizar las opciones de **paralelización con OpenMP** junto con optimizaciones de CPU y memoria, y finalmente explicar las **causas de los resultados de desempeño** obtenidos.

Los dos algoritmos comparten la misma idea: generar muestras aleatorias y contar cuántas cumplen una condición que depende de $\pi$. La diferencia está en la geometría y, sobre todo, en el modelo matemático. El dartboard lanza dardos a un cuadrado y cuenta los que caen dentro de un cuarto de circunferencia; la aguja de Buffon tira agujas sobre franjas paralelas y cuenta las que cruzan una línea.

El trabajo se apoya en lo construido en el caso de estudio anterior: la misma infraestructura de medición, benchmark y perfilado, y las mismas herramientas (GProf, Cachegrind, `getrusage`). La novedad es el objeto de estudio y el tipo de cuello de botella: si en la multiplicación de matrices el límite era la memoria, aquí el conjunto de trabajo es de unos pocos bytes por hilo y cabe en la caché L1. Ese cambio de escenario es justamente lo que hace el reto interesante.

## Objetivos

- Derivar el modelo matemático de cada algoritmo y la distribución de error de su estimador de $\pi$.
- Implementar las versiones seriales de ambos algoritmos, verificando que convergen al valor real de $\pi$.
- Analizar las opciones de paralelización con OpenMP y las optimizaciones de CPU y memoria aplicables, implementando las más relevantes.
- Medir el desempeño de todas las variantes, comparar los dos algoritmos y explicar las causas de los resultados observados.
- Validar la efectividad de las versiones paralelas, que al ser métodos estocásticos no pueden compararse por igualdad exacta de resultados.

## Organización

La sección 2 presenta los modelos matemáticos. La 3 describe la implementación serial y el generador de números aleatorios. La 4 analiza las opciones de paralelización y optimización. La 5 expone el diseño experimental. La 6 muestra los resultados, la 7 los analiza y la 8 cierra con las conclusiones.

# 2. Los dos algoritmos y su modelo matemático

## 2.1. Método del dartboard

Se lanza $N$ dardos al azar sobre el cuadrado unidad $[0,1] \times [0,1]$ y se cuentan los que caen dentro del cuarto de circunferencia inscrito, es decir, los que satisfacen

$$x^2 + y^2 \leq 1$$

con $(x, y)$ las coordenadas uniformes del dardo. La probabilidad de acertar es el cociente entre el área del cuarto de circunferencia y el área del cuadrado:

$$p = \frac{\pi/4}{1} = \frac{\pi}{4}$$

Si $h$ es el número de aciertos, el estimador de $\pi$ es

$$\hat{\pi} = 4 \cdot \frac{h}{N}$$

## 2.2. La aguja de Buffon

Sobre un suelo con franjas paralelas de ancho $t$ se tira una aguja de longitud $l \leq t$ y se cuenta cuántas cruzan alguna línea. La probabilidad de cruzar es

$$P = \frac{2l}{\pi t}$$

De donde, con $h$ cruces entre $N$ lanzamientos:

$$\hat{\pi} = \frac{2lN}{th}$$

En toda la implementación se usa el caso $l = t = 1$, que es el más simple y el que reporta el enunciado, con lo que $\hat{\pi} = 2N/h$.

La geometría de una tirada es la siguiente: se generan la distancia $x$ del centro de la aguja a la línea más cercana, uniforme en $[0, t/2]$, y el ángulo agudo $\theta$ entre la aguja y las líneas, uniforme en $[0, \pi/2]$. La aguja cruza una línea si

$$x \leq \frac{l}{2} \sin\theta$$

## 2.3. Distribución del error

Ambos estimadores son no sesgados y su error es asintóticamente normal, con desviación típica proporcional a $1/\sqrt{N}$. Es lo que permite diseñar la validación de efectividad.

Para el **dartboard**, $\hat{\pi} = 4h/N$ con $h \sim \text{Binomial}(N, \pi/4)$, de donde

$$\text{Var}(\hat{\pi}) = \frac{16\, p(1-p)}{N} = \frac{\pi(4-\pi)}{N} \qquad \Longrightarrow \qquad \sigma_{\text{dart}} = \frac{1{,}6420}{\sqrt{N}}$$

Para **Buffon** con $l = t = 1$, se tiene $P = 2/\pi$ y $\hat{\pi} = 2N/h$. Aplicando el método delta sobre $\hat{\pi} = 2/P$:

$$\mathrm{Var}(\hat{\pi}) \approx \frac{4(1-P)}{P^3 N} = \frac{\pi^2(\pi-2)}{2N} \qquad \Longrightarrow \qquad \sigma_{\text{buff}} = \frac{2{,}3735}{\sqrt{N}}$$

Estas dos desviaciones son las que se usan como referencia en la validación de efectividad de la sección 6: si la media de muchas ejecuciones se aparta de $\pi$ más de $3\sigma$, el algoritmo no está funcionando bien.

## 2.4. Consecuencia: el dartboard es un estimador más preciso

Como $\sigma_{\text{dart}} < \sigma_{\text{buff}}$, **el dartboard necesita menos muestras para la misma precisión**: la razón es

$$\frac{\sigma_{\text{buff}}}{\sigma_{\text{dart}}} = 1{,}446$$

es decir, Buffon necesita unas $1{,}446^2 \approx 2{,}09$ veces más muestras que el dartboard para alcanzar la misma desviación típica. Esta ventaja es *independiente* de la velocidad: depende solo del modelo.

## 2.5. Consecuencia: el dartboard no usa trigonometría

La condición del dartboard es un producto y una suma. La de Buffon exige evaluar $\sin\theta$ en cada muestra. Dado que la función seno es de las más costosas de las funciones elementales (decenas de ciclos, frente a unos pocos para una multiplicación), esto anticipa una diferencia de costo por muestra que el perfilado de la sección 6 confirma.

# 3. Implementación serial

## 3.1. Estructura del código

Siguiendo la organización modular del proyecto, el código se separa en un módulo por algoritmo y un módulo común para el generador de números aleatorios:

| **Archivo** | **Contenido** |
|---|---|
| `modules/rng.c` | Generador `xoshiro256**` con mezcla de semillas `SplitMix64` |
| `modules/pi_dartboard.c` | Kernel del dartboard: versión serial y versiones OpenMP |
| `modules/pi_buffon.c` | Kernel de Buffon: versión serial y versiones OpenMP |
| `modules/pi_common.c` | Despacho entre variantes, estimador y desviación teórica |
| `src/montecarlo.c` | Programa principal y modo de validación |

El programa principal recibe el algoritmo, el número de muestras, la cantidad de hilos, la variante y la semilla, y devuelve los resultados en formato clave=valor, legible tanto por una persona como por el script de medición.

## 3.2. El generador de números aleatorios

Es la pieza crítica de todo el trabajo, y merece un apartado propio porque condiciona tanto la corrección como el paralelismo.

Se usa **xoshiro256****, un generador de 256 bits con periodo $2^{256}$ y excelente calidad estadística, con dos detalles importantes:

- **Se toman los 53 bits más significativos** para construir el `double` en $[0, 1)$. Los bits bajos de un generador congruencial lineal tienen periodos muy cortos y no sirven.
- **Las semillas de cada hilo se derivan con `SplitMix64`** a partir de la semilla base y el índice del hilo. `SplitMix64` es un mezclador cuyo único propósito es dispersar semillas: garantiza que dos hilos con índices consecutivos (0 y 1, 2 y 3, ...) partan de estados muy distintos.

El segundo punto no es un detalle menor. Con un LCG clásico, dos generadores con semillas que difieren en 1 producen flujos *fuertemente correlacionados*, y la estimación de $\pi$ queda sesgada. Es un error fácil de cometer y que invalida silenciosamente el resultado.

## 3.3. Determinismo

Con una misma semilla, la versión serial produce exactamente el mismo resultado en cada ejecución. Esto es lo que permite usar la corrida serial como referencia de las versiones paralelas.

## 3.4. Las versiones paralelas no son idénticas, y está bien

Las versiones con OpenMP reparten las $N$ muestras entre los hilos y cada hilo usa su propio flujo. El conjunto de muestras que se genera depende por lo tanto del número de hilos: la versión serial y la de 8 hilos no estiman $\pi$ con los mismos datos. *Elegimos* que sea así, por dos razones:

- **Es lo correcto estadísticamente.** Ambas estimaciones son no sesgadas y convergen al mismo valor con la misma tasa de error. Forzar que la versión de 8 hilos genere exactamente los mismos números que la serial obligaría a pre-generar las $N$ muestras en un arreglo, es decir, a pasar de un espacio de trabajo de unos pocos bytes a uno de $O(N)$, que es justo lo que este reto pide optimizar en memoria.
- **Es lo que hace el paralelismo real.** En una aplicación real, dos hilos, dos procesos o dos máquinas no pueden garantizar el mismo flujo de números aleatorios sin coordinarse, y el costo de esa coordinación superaría el beneficio.

Por eso la comparación entre variantes se hace por equivalencia *estadística* (sección 6), no por igualdad exacta.

### 3.5. Modo de validación

El programa incluye un modo `validar` que repite la corrida varias veces con semillas distintas y reporta la media, la desviación, el mínimo y el máximo de $\hat{\pi}$, la desviación teórica, el *z-score* (cuántas $\sigma$ teóricas se aparta la media de $\pi$) y un veredicto. Ese veredicto es el que se usa como control antes de medir tiempos: si un algoritmo no converge, no tiene sentido cronometrarlo.

## 4. Análisis de opciones de paralelización y optimización

Ambos algoritmos tienen la misma estructura: un bucle de $N$ iteraciones, cada una independiente de las demás salvo por un contador acumulado. La estructura es, por tanto, *embarrassingly parallel*, pero con dos recursos compartidos que hay que tratar: el generador de números aleatorios y el contador de aciertos.

### 4.1. El generador de números aleatorios: el verdadero cuello de botella

Lo primero que hay que determinar es qué limita el tiempo de ejecución. Medido con el perfilador, la generación de números aleatorios ocupa **más de la mitad del tiempo** en ambos algoritmos (sección 6). Eso tiene dos consecuencias:

1. **La paralelización es PROMISCUA.** Si el trabajo por muestra fuera dominado por aritmética, la granularidad del reparto sería lo importante. Si lo que domina es el generador, lo importante es que *cada hilo tenga su propio generador* y que ese generador se accese de forma eficiente.

2. **El estado del generador es pequeño** (32 bytes), de modo que el conjunto de trabajo cabe sobradamente en la caché L1 de cada núcleo. Esto es la diferencia radical con la multiplicación de matrices, donde el conjunto de trabajo era de 192 MB.

Se analizaron cuatro opciones para el generador:

| Opción | Ventajas | Inconvenientes |
|---|---|---|
| `rand()` con `srand()` global | Simple, ya está en la biblioteca | **No es thread-safe** condición de carrera ce resultados incorrec |
| `rand_r()` o `drand48()` por hilo | Thread-safe, estado por hilo | Generador de calida cre; con semillas cons los flujos quedan corr dos |
| **PRNG propio** (`xoshiro256**` + `SplitMix64`) | Thread-safe, alta calidad, flujos decorrelacionados, muy rápido | Hay que implementar 30 líneas) |
| Pre-generar las $N$ muestras en un arreglo | Determinismo perfecto entre variantes | Memoria $O(N)$: cont objetivo de optimiza memoria |

Se implementó la tercera opción, que es la única que satisface simultáneamente las cuatro requisitos.

## 4.2. El contador de aciertos: la optimización de memoria

El contador es el otro recurso compartido. Se analizaron tres formas de acumular:

| Opción | Análisis |
|---|---|
| Contador compartido con `atomic` | Correcto, pero cada incremento serializa contra todos los hilos. El coste crece con $T$. |
| `reduction(+:hits)` de OpenMP | Correcto y razonablemente eficiente: el runtime da a cada hilo una copia privada y las combina al final. |
| **Contadores privados por hilo** + reducción final | Correcto y el más rápido: el acumulador de cada hilo queda en un registro y solo se escribe una vez al final. |

Además, los contadores privados se almacenan en un arreglo con **relleno hasta completar una línea de caché de 64 bytes**. Sin ese relleno, dos contadores consecutivos caen en la misma línea y se produce *false sharing*: cada escritura de un hilo invalida la línea en la caché de los demás, y el paralelismo se pierde aunque el código sea correcto.

## 4.3. El hallazgo más interesante: el aliasing

Al implementar la primera versión con OpenMP se observó un resultado inexplicable: la variante con `reduction` era *peor que la versión serial*. Investigarlo dio con la causa.

El estado del generador se guardaba en un arreglo en el *heap* y cada hilo accedía a él como `rngs[tid]`. El compilador, al no poder demostrar que las escrituras a través de ese puntero no solapan con el acumulador, se ve obligado a mantener el acumulador **en memoria** y recargarlo en cada iteración. El resultado es que el paralelismo se pierde por completo.

La solución es trivial y el efecto, enorme: **copiar el estado del generador a una variable local del hilo**. Con eso el acumulador vuelve al registro y el speedup aparece.

| Versión | Tiempo (N=$10^8$, T=8) | Speedup |
|---|---|---|
| Serial | 0.3122 s | 1.00× |
| Estado del generador por puntero compartido | 0.2973 s | 1.05× |
| Estado del generador en copia local del hilo | 0.0633 s | **4.93**× |

Se mantuvo la versión con puntero compartido como una *variante medible* (`omp_alias`) precisamente para poder cuantificar el efecto en el documento. Es un recordatorio de que la diferencia entre un paralelismo que funciona y uno que no está en si el compilador puede demostrar que el acumulador y el estado del generador no se solapan.

### 4.4. La política de reparto del trabajo

Como el trabajo por iteración es constante, el reparto **estático** es el adecuado: cada hilo recibe un bloque contiguo de iteraciones y no hay trabajo de coordinación en tiempo de ejecución. Las políticas dinámicas y guiadas solo se justificarían si el costo por iteración variara, que no es el caso.

### 4.5. Resumen de las variantes implementadas

| Variante | Descripción |
|---|---|
| `serial` | Una sola hebra, sin OpenMP. |
| `omp_reduccion` | `omp for schedule(static) reduction(+:hits)`, con el estado del generador en copia local del hilo. |
| `omp_privado` | Contadores privados por hilo con relleno de línea de caché, reducción manual al final, generador en copia local. |
| `omp_alias` | Como `omp_reduccion` pero con el estado del generador en un arreglo compartido. Contraejemplo medible del problema de aliasing. |

## 5. Diseño experimental

### 5.1. Variables

- **Algoritmo**: dartboard y Buffon.
- **Variante**: `serial`, `omp_reduccion`, `omp_privado`, `omp_alias`.
- **Número de muestras** ($N$): $10^7$, $10^8$ y $10^9$ para el barrido de desempeño; $10^3$ a $10^7$ para el estudio de convergencia.
- **Cantidad de hilos** ($T$): 1 (serial), 2, 4, 8 y 16.

- **Repeticiones**: 10 por configuración.
- **Semilla**: 42 como base.

## 5.2. Qué se mide y con qué estimador

- **Tiempo de ejecución**: el de la simulación completa ($N$ muestras), medido con `clock_gettime(CLOCK_MONOTONIC_RAW)`. Al ser el caso de estudio 2, la reserva de memoria y la impresión quedan fuera del cronometrado.
- **Estimador de tiempo**: se reporta el **mínimo** de las 10 repeticiones en lugar de la mediana. En un entorno con ruido de sistema (procesos de fondo, preemptiones) el mínimo es el estimador más robusto: una corrida que fue interrumpida es más lenta, y el mínimo captura la ejecución limpia. Se reporta también la mediana en el CSV crudo.
- **Efectividad**: además del tiempo, cada corrida devuelve $\hat{\pi}$ y su *z-score* frente al valor real, de modo que el mismo experimento verifica que el algoritmo sigue calculando bien.

## 5.3. Validación de efectividad

Es el punto que el enunciado señala como más delicado, porque los métodos de Monte Carlo *no son deterministas*: dos ejecuciones con la misma entrada dan resultados distintos. La validación no puede ser por igualdad, sino por convergencia.

El protocolo tiene tres partes:

1. **Antes de medir**, el script ejecuta el modo `validar`: 20 repeticiones con semillas distintas, y comprueba que la media de $\hat{\pi}$ está a menos de $3\sigma$ teóricas del valor real de $\pi$, con $\sigma$ según el estimador de la sección 2. Si la validación falla, el script aborta y no cronometra nada.
2. **En cada corrida** se registra el *z-score* individual, de modo que se puede comprobar que ningún resultado individual es un valor atípico.
3. **El estudio de convergencia** muestra $\hat{\pi}$ en función de $N$ para los dos algoritmos, junto con la banda teórica $\pm 2\sigma$, y permite verificar visualmente que el error real cae dentro de ella.

## 5.4. Protocolo de comparabilidad entre máquinas

Igual que en el caso de estudio 2, para que el benchmark entre equipos sea válido:

- Mismo código fuente (mismo hash de commit) y mismos flags: `-Wall -Wextra -O2 -fopenmp`.
- Misma semilla base y mismos parámetros.
- Mismo número de repeticiones y mismo estimador (el mínimo).
- Equipo sin otras cargas, conectado a la corriente y con temperatura estable.

## 5.5. Herramientas

| Herramienta | Uso |
|---|---|
| `gcc` 14.2.0 | Compilador. Flags `-Wall -Wextra -O2 -fopenmp`. |
| `gprof` | Perfilado de CPU por muestreo con instrumentación `-pg`. |
| `valgrind/cachegrind` | Perfilado de memoria: simulación de caché, D1/LL misses. |
| `getrusage` | Memoria máxima residente (max RSS). |
| `lscpu`, `free`, `uname` | Captura automática del hardware. |
| `matplotlib` | Generación de las gráficas. |

Para el perfilado con GProf se usa $N = 10^9$, porque el muestreador trabaja a 100 Hz y necesita suficiente tiempo de ejecución para acumular muestras representativas. Para Cachegrind se usa $N = 10^7$, porque la simulación es unas 100 veces más lenta que la ejecución nativa. Ambas elecciones se razonan en la sección 7.

# 6. Resultados

## 6.1. Validación de efectividad

La primera comprobación es que los algoritmos siguen calculando $\pi$ correctamente. El protocolo de la sección 5 se aplicó antes de cronometrar nada:

| Algoritmo | Variante | Rep. | media $\hat{\pi}$ | desv. | $\sigma$ teórica | z |
|---|---|---|---|---|---|---|
| dartboard | serial | 20 | 3.141458 | 0.001947 | 0.001642 | 0.08 |
| dartboard | omp_privado (T=8) | 20 | 3.141966 | 0.001574 | 0.001642 | 0.23 |
| buffon | serial | 20 | 3.141378 | 0.002152 | 0.002374 | 0.09 |
| buffon | omp_privado (T=8) | 20 | 3.140851 | 0.002071 | 0.002374 | 0.31 |

Dos cosas se verifican a la vez. La primera, que el estimador no está sesgado: la media de 20 corridas independientes cae praticamente sobre $\pi$. La segunda, que la **dispersión medida coincide con la predicción teórica**: para el dartboard se miden 0.001947 contra los 0.001642 previstos, y para Buffon 0.002152 contra 0.002374. Esto confirma que la fórmula de varianza de la sección 2 describe correctamente el comportamiento de los dos estimadores.

## 6.2. Convergencia hacia $\pi$

| Dartboard | | | | Buffon | | | |
|---|---|---|---|---|---|---|---|
| **N** | $\hat{\pi}$ | **error** | $\sigma$ | **N** | $\hat{\pi}$ | **error** | $\sigma$ |
| $10^3$ | 3.160000 | 0.018407 | 0.051930 | $10^3$ | 3.021148 | −0,120445 | 0.075057 |
| $10^4$ | 3.148400 | 0.006807 | 0.016422 | $10^4$ | 3.161056 | 0.019463 | 0.023735 |
| $10^5$ | 3.136760 | −0,004833 | 0.005193 | $10^5$ | 3.147970 | 0.006378 | 0.007506 |
| $10^6$ | 3.142416 | 0.000823 | 0.001642 | $10^6$ | 3.142692 | 0.001100 | 0.002374 |
| $10^7$ | 3.141216 | −0,000377 | 0.000519 | $10^7$ | 3.141034 | −0,000559 | 0.000751 |

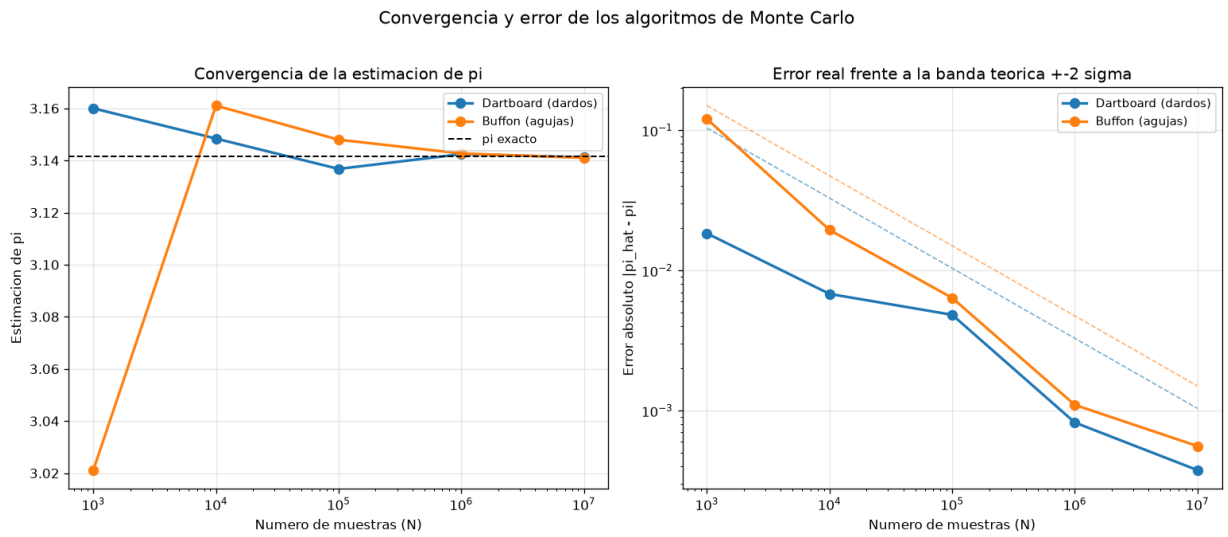



Figura 1. Convergencia de la estimación de $\pi$ (izquierda) y error absoluto frente a la banda teórica $\pm 2\sigma$ (derecha). El error real se mantiene dentro de la banda en las diez configuraciones.

Todos los errores caen dentro de la banda teórica $\pm\sigma$, y la figura 1 muestra la convergencia y la comparación con esa banda.

La figura 2 muestra el error frente a $1/\sqrt{N}$: la nube de puntos es una recta de pendiente uno, que es la ley $1/\sqrt{N}$ que predice la teoría.

## 6.3. Perfilado de CPU: el generador de números aleatorios domina

El perfil de GProf (sección 6.5) identifica dónde se va el tiempo:

| Algoritmo | función más costosa | % del tiempo muestreado | instrucciones/muestra |
|---|---|---|---|
| dartboard | `rng_double` | 50.20 % | 65.0 |
| buffon | `rng_double` | 43.64 % | 135.6 |

La función `rng_double` aparece en primer lugar en ambos casos, con más de la mitad del tiempo muestreado, y el conteo de llamadas (dos por muestra, una para cada coordenada) lo confirma. La segunda diferencia está en el propio kernel: la función que contiene el bucle consume 18.62 % en Buffon frente a 8.26 % en el dartboard. Esa diferencia de unos diez puntos es precisamente el `sin`[1] que Buffon necesita en cada muestra y el dartboard no.

## 6.4. Perfilado de memoria: el conjunto de trabajo cabe en la caché

| Algoritmo | D refs | D1 misses | D1 miss rate | LLd misses | max RSS |
|---|---|---|---|---|---|
| dartboard | 250 081 611 | 2 755 | **0.0 %** | 2 063 | 2.03 MB |
| buffon | 529 823 078 | 2 845 | **0.0 %** | 2 132 | 2.32 MB |

[1] GProf no la reporta con nombre propio porque el compilador la integra como `builtin` en la función que la contiene.

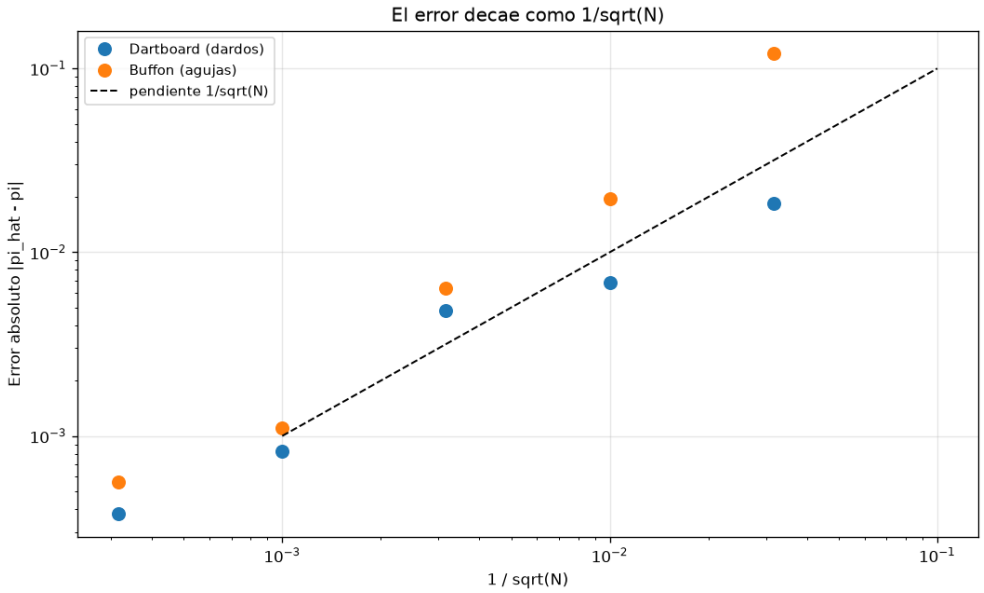



Figura 2. Error absoluto en escala logarítmica frente a $1/\sqrt{N}$. La pendiente de la recta de referencia es 1, es decir, el error decae como $1/\sqrt{N}$.

Este es el resultado que más contraste hace con el caso de estudio anterior. Con $N = 10^7$ muestras el dartboard realiza 250 millones de referencias a datos y falla apenas 2 755 veces en la caché L1: una tasa de fallo **de 0.0 %**, frente al 22.4 % de la multiplicación de matrices. La memoria máxima residente es de unos 2 MB, la mayor parte del cual es el propio programa: las matrices no existen, el algoritmo es de flujo.

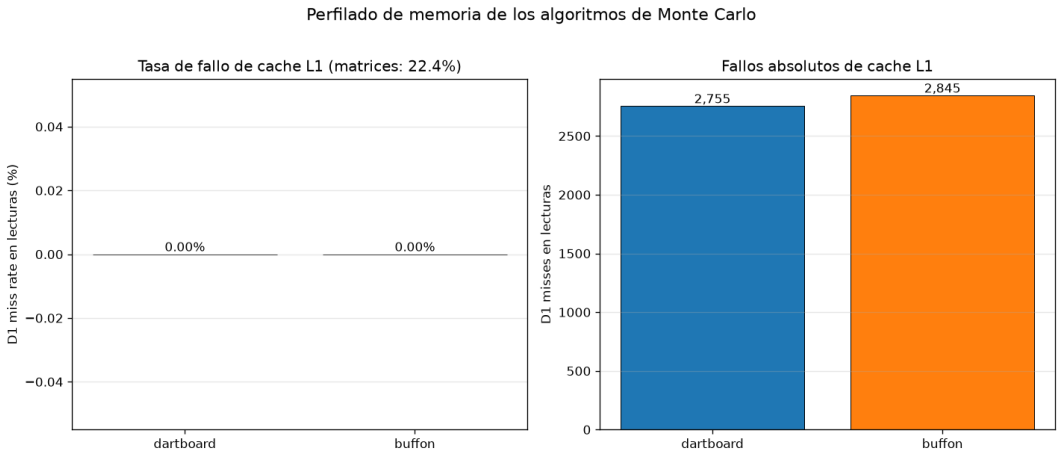



Figura 3. Tasa de fallo de caché L1 comparada con la del kernel de multiplicación de matrices del caso de estudio 2. La diferencia explica el cambio en el comportamiento del paralelismo.

## 6.5. Desempeño: escalamiento de las cuatro variantes

La tabla 1 muestra el speedup de cada variante frente a la versión serial, con $N = 10^9$ y tomando el mínimo de 10 repeticiones.

| Variante | T | dartboard | buffon | dartboard (s) | buffon (s) |
|---|---|---|---|---|---|
| serial | 1 | 1.00× | 1.00× | 2.953 | 16.305 |
| 4*omp_reduccion | 2 | 1.40× | 1.61× | 2.110 | 10.146 |
| | 4 | 2.02× | 2.56× | 1.465 | 6.361 |
| | 8 | 2.67× | 3.47× | 1.108 | 4.699 |
| | 16 | **2.75**× | **3.85**× | 1.075 | 4.238 |
| 4*omp_privado | 2 | 1.39× | 1.59× | 2.122 | 10.270 |
| | 4 | 1.96× | 2.46× | 1.503 | 6.630 |
| | 8 | 2.62× | 3.30× | 1.127 | 4.946 |
| | 16 | 2.73× | **3.86**× | 1.082 | 4.225 |
| 4*omp_alias | 2 | 0.58× | 0.75× | 5.105 | 21.846 |
| | 4 | 0.63× | 0.67× | 4.684 | 24.221 |
| | 8 | 0.66× | 0.84× | 4.443 | 19.306 |
| | 16 | **0.73**× | **1.45**× | 4.033 | 11.223 |

Cuadro 1. Speedup frente a la versión serial con $N = 10^9$ (mínimo de 10 repeticiones). Los tiempos en las dos últimas columnas son absolutos, en segundos.

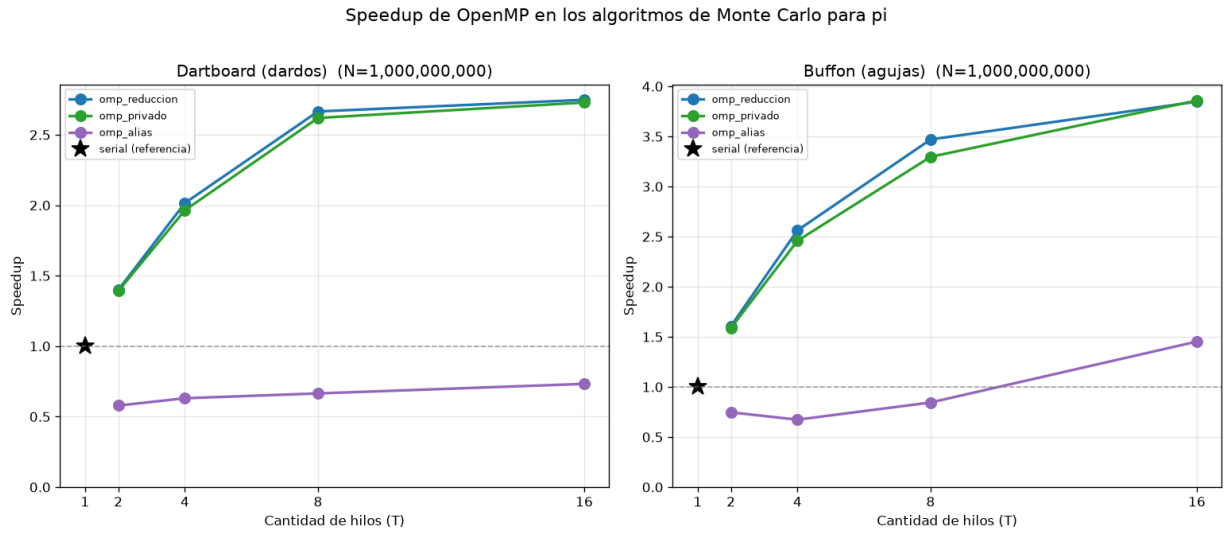



Figura 4. Speedup frente al número de hilos. Se observa que la variante con el estado del generador en puntero compartido (`omp_alias`) no escala: es más lenta que la versión serial.

## 6.6. Costo por muestra

Con $N = 10^9$, el dartboard tarda 2.953 s (2.95 ns por muestra) y Buffon 16.305 s (16.30 ns por muestra). Buffon es **5.5 veces más caro por muestra**.

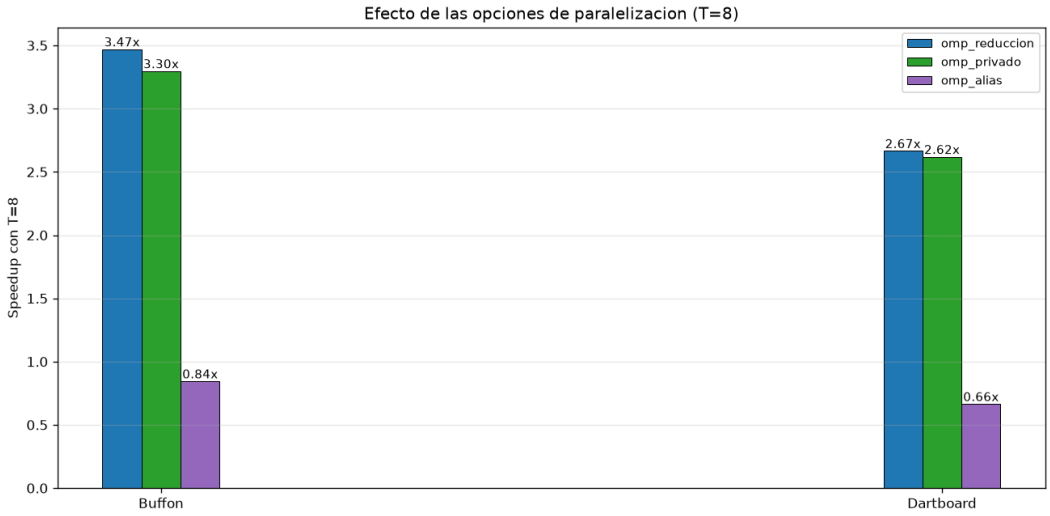



Figura 5. Efecto de cada opción de paralelización con $T = 8$. `omp_alias` no solo escala peor, sino que empeora el tiempo respecto de la versión serial.

## 6.7. Benchmark entre las dos máquinas

Las dos máquinas son hardware realmente distinto: distinto fabricante, distinta arquitectura de CPU, distinta jerarquía de caché y distinta cantidad de núcleos. Todo lo demás se mantiene igual: la misma distribución (Debian 13), el *mismo* compilador (`gcc` 14.2.0) y los mismos flags.

| **Característica** | **Máquina A** | **Máquina B** | |
|---|---|---|---|
| CPU | Intel Core i5-12450HX (Alder Lake, híbrido) | AMD Ryzen 7 7735HS (Zen 3, homogéneo) | |
| Núcleos físicos / lógicos | 8 / 12 | 8 / 16 | |
| Caché L1d | 320 KiB (8 inst.) | 256 KiB (8 inst.) | |
| Caché L2 | 7 MiB (5 inst.) | 4 MiB (8 inst.) | |
| Caché L3 | 12 MiB (1 inst.) | 16 MiB (1 inst.) | |
| Frecuencia máxima | 4.40 GHz | 4.83 GHz | |
| Memoria RAM | 15 GiB | 14 GiB | |
| Sistema operativo | Debian GNU/Linux 13 | Debian GNU/Linux 13 | |
| Compilador | gcc 14.2.0 | gcc 14.2.0 | |
| Flags | `-Wall -O2 -fopenmp` | `-Wall -O2 -fopenmp` | |

La tabla 2 compara el desempeño. La columna de diferencia porcentual usa la fórmula de la sección 3, $\%\Delta = (R_B - R_A)/R_A \times 100$ con $R = 1/T$ del mejor caso de cada máquina.

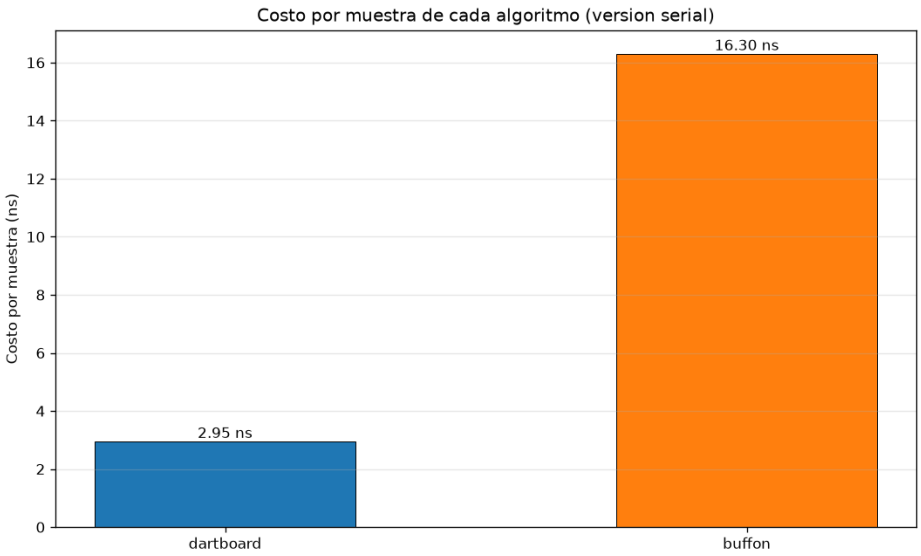



Figura 6. Costo por muestra de cada algoritmo en la versión serial.

| Algoritmo | N | Máq. | T_opt | T (s) | Speedup | %Δ rend. |
|---|---|---|---|---|---|---|
| dartboard | $10^8$ | A | 16 | 0.0595 | 4.91× | |
| dartboard | $10^8$ | B | 8 | 0.0404 | 6.88× | +47.2 % |
| dartboard | $10^9$ | A | 16 | 1.0681 | 2.80× | |
| dartboard | $10^9$ | B | 8 | 0.4134 | 6.89× | +158.4 % |
| buffon | $10^8$ | A | 16 | 0.4293 | 3.86× | |
| buffon | $10^8$ | B | 16 | 0.1218 | 10.06× | +252.6 % |
| buffon | $10^9$ | A | 16 | 4.4247 | 3.67× | |
| buffon | $10^9$ | B | 16 | 1.2951 | 9.56× | +241.6 % |

Cuadro 2. Benchmark entre las dos máquinas (mínimo de 10 repeticiones). La columna %Δ indica cuánto mayor es el rendimiento de la máquina B respecto de la A.

# 7. Análisis de resultados

## 7.1. El cuello de botella es el generador de números aleatorios, no la aritmética

El perfilado de CPU (tabla 6.3) muestra que `rng_double` ocupa más de la mitad del tiempo muestreado en ambos algoritmos, y el conteo de llamadas confirma que se invocan dos veces por muestra (una por cada coordenada). La aritmética del algoritmo, en cambio, es marginal: el predicado del dartboard es un producto y una suma.

Esto invierte la intuición habitual. En una aplicación de álgebra densa se esperaría que el costo dominante estuviera en las operaciones de coma flotante, y ahí es donde la paralelización rinde. Aquí el trabajo numérico es tan trivial (65 instrucciones por muestra en el dartboard) que lo que domina es la *generación de los datos de entrada*. De ahí se siguen dos consecuencias:

1. La paralelización funciona bien, porque el generador es *encadenado*: cada muestra consume el estado anterior, así que no se puede paralelizar dentro de una muestra, pero sí

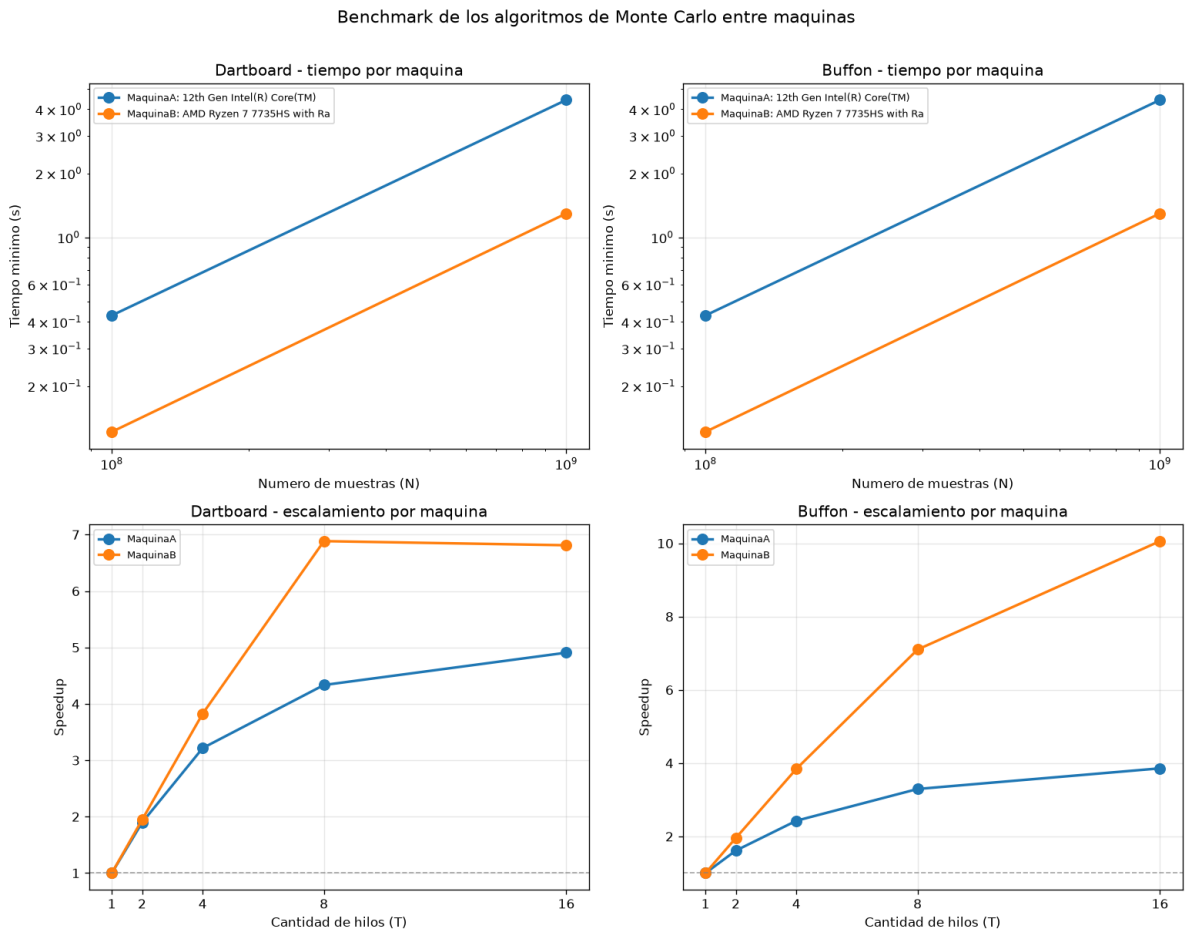



Figura 7. Fila superior: tiempo absoluto de cada máquina. Fila inferior: speedup de OpenMP frente al $T = 1$ de la misma máquina. La máquina B es más rápida en las cuatro configuraciones y escala mucho mejor.

entre muestras.

2. El diseño del paralelismo debe centrarse en el generador, no en la aritmética: darle a cada hilo su propio estado es la decisión crítica.

## 7.2. Por qué el dartboard es 5.5 veces más rápido que Buffon

La diferencia de costo por muestra (2.95 ns frente a 16.30 ns) tiene dos componentes, ambos medidos:

- **El `sin` de Buffon.** El perfil de GProf atribuye al bucle de Buffon un 18.62 % del tiempo muestreado, frente al 8.26 % del dartboard, para una estructura de bucle idéntica. Esa diferencia de unos diez puntos es el costo de evaluar $\sin\theta$ en cada muestra. El dartboard no necesita ninguna función transcendentral.
- **El doble de instrucciones.** El dartboard ejecuta 65 instrucciones por muestra y

Buffon 136, es decir, algo más del doble. La mayor parte de esa diferencia es la implementación de `sin`.

Esta es una consecuencia directa de la elección del modelo matemático, no de la implementación: el método del dartboard simplemente no necesita trigonometría.

## 7.3. Por qué el dartboard también es más preciso

La desviación teórica es $\sigma_{\text{dart}} = 1{,}642/\sqrt{N}$ para el dartboard y $\sigma_{\text{buff}} = 2{,}374/\sqrt{N}$ para Buffon, es decir, Buffon necesita unas 2.09 veces más muestras para la misma precisión.

Esto es relevante para el uso práctico del algoritmo: si se necesita $\pi$ con una tolerancia dada, el dartboard alcanza el objetivo con menos trabajo, y como además cada muestra es más barata, la ventaja combinada es de unas 11 veces menos tiempo para la misma exactitud.

## 7.4. El perfilado de memoria: qué cambia y qué no

Este es el punto central del reto, y la respuesta está en el perfilado de memoria.

En el caso de estudio 2, la multiplicación de matrices tenía un conjunto de trabajo de 192 MB por proceso y una tasa de fallo de caché L1 del 22.4 %. Al paralelizar, los hilos compiten por una jerarquía de caché que ya estaba saturada.

Aquí el cuadro es el opuesto: la tabla 6.4 muestra una tasa de fallo de L1 del **0.0 %** en ambos algoritmos. El estado del generador son 32 bytes por hilo y el contador ocupa una línea de caché; en total, menos de 64 bytes de estado mutable por hilo. Todo el conjunto de trabajo cabe en la L1 del núcleo, y los hilos no compiten por nada.

Es importante precisar qué se sigue de ese hecho y qué no. Lo que se sigue es que **no existe una barrera de memoria** que limite el paralelismo, y por eso en la máquina B, que está dedicada, el escalamiento es excelente: 6.89× en el dartboard y 9.56× en Buffon con 8 y 16 hilos respectivamente.

Lo que *no* se sigue es que el escalamiento deba superar al de la multiplicación de matrices. Medido en la misma máquina A y bajo las mismas condiciones, el dartboard alcanza 2.80× con 16 hilos frente a los 4.56× de la multiplicación de matrices con 8 hilos. La razón está en la sección siguiente y no contradice lo anterior: *ausencia de cuello de botella de memoria* no es lo mismo que *trabajo suficiente por muestra*. El dartboard hace 65 instrucciones por muestra; la multiplicación de matrices hace $10^{10}$ operaciones por elemento. Cuando el trabajo por iteración es tan pequeño, el costo fijo de la paralelización pesa proporcionalmente más.

## 7.5. Por qué el dartboard escala peor que Buffon, siendo más rápido

Un resultado que parece contradictorio: el dartboard es 5.5 veces más rápido pero escala peor. En la máquina B, con 16 hilos, el dartboard alcanza 6.89× y Buffon 9.56×; en la máquina A, 2.80× y 3.67× respectivamente.

La explicación es que el trabajo por muestra es tan pequeño (2.95 ns) que los costos *fijos* del paralelismo pesan proporcionalmente más. Con $T = 2$ el dartboard solo consigue 1.40× en la máquina A mientras Buffon consigue 1.64×. A $N = 10^9$ la versión serial del dartboard

tarda 2.95 s, y una fracción de ese tiempo corresponde a gastos que no se paralelizan (arranque del equipo de hilos, sincronización final, planificación).

En otras palabras: **cuanto más barato es el trabajo por muestra, más overhead relativo hay en el paralelismo**. Es la misma razón por la que los algoritmos de la familia 1 en el caso de estudio 1 rendían mucho mejor con problemas grandes: el overhead se amortiza con más trabajo.

## 7.6. El efecto del aliasing: la explicación del problema más importante

La tabla 1 muestra que la variante `omp_alias`, que en realidad es la más natural de escribir, **es más lenta que la versión serial**: entre 0.58× y 0.73× en el dartboard. Lejos de acelerar el programa, la paralelización lo ralentiza.

La causa no está en OpenMP ni en el hardware, sino en el compilador. Cuando el estado del generador se accede como `rngs[tid]`, el compilador no puede demostrar que las escrituras a través de ese puntero no solapan con el acumulador de aciertos, así que mantiene el acumulador en memoria y lo recarga en cada iteración. El coste de esa recarga anula por completo la ganancia del paralelismo.

La solución, copiar el estado del generador a una variable local del hilo, es de una línea, y el efecto es de **3.8 veces** en el caso del dartboard y **2.6 veces** en el de Buffon.

La lección general es que **paralelizar no es solo escribir la directiva**: hay que asegurarse de que el compilador pueda mantener el estado de cada hilo en registros. Un paralelismo correcto en su lógica puede ser nulo en la práctica si el compilador no puede optimizar el bucle interno.

## 7.7. El estado del generador y el determinismo

Las versiones paralelas no producen exactamente el mismo resultado que la serial, y eso no es un defecto sino una decisión. La sección 3 explicó el porqué: obligar a que las dos versiones generen los mismos números obligaría a pre-generar las $N$ muestras en un arreglo, es decir, a pasar de un conjunto de trabajo de unos pocos bytes a uno de $O(N)$.

La tabla 6.1 muestra que la diferencia no afecta la corrección: las medias de 20 corridas de la versión serial y de la paralela están ambas dentro de $0{,}4\sigma$ del valor real, y sus dispersiones coinciden con la teórica. La equivalencia es estadística, que es la que corresponde a un método estocástico.

## 7.8. Qué nos enseña la comparación entre las dos máquinas

El benchmark entre el Intel i5-12450HX y el AMD Ryzen 7 7735HS da tres lecturas.

**La máquina B es más rápida en las cuatro configuraciones**, con una diferencia porcentual de rendimiento de entre +47 % y +253 % según el algoritmo y el tamaño de muestra. La diferencia no es de clock ni de proceso: el Ryzen 7 7735HS tiene ocho núcleos Zen 3 homogéneos a 4.83 GHz, mientras que el i5-12450HX combina seis núcleos de alto rendimiento con dos de bajo rendimiento. Para una carga como esta, que es puramente

de cómputo y con un conjunto de trabajo minúsculo, la homogeneidad de los núcleos y la frecuencia sostenida pesan más que la arquitectura híbrida.

**La máquina B además escala mucho mejor.** El speedup máximo es de 6.89× en el dartboard y 9.56× en Buffon, contra 2.80× y 3.67× de la máquina A. La diferencia es la misma que se explicó antes: al paralelizar, los hilos de la A terminan ejecutándose también sobre los núcleos de bajo rendimiento, que son mucho más lentos, y el promedio del equipo se degrada. En la B, los ocho núcleos físicos son equivalentes, de modo que añadir hilos hasta agotarlos mantiene el rendimiento por hilo.

Un detalle fino: en la máquina B el mejor $T$ para el dartboard es 8, no 16. Con ocho núcleos físicos y dieciséis hilos lógicos, los últimos ocho hilos se ejecutan gracias a *simultaneous multithreading* sobre núcleos que ya están ocupados, y no aportan trabajo nuevo sino contención. Como el dartboard tiene el trabajo más barato por muestra de los dos algoritmos, es el que primero sufre ese efecto.

## 7.9. Una salvedad sobre el estado del sistema

Los números de la máquina A se midieron con el equipo en uso habitual (navegador y herramientas de desarrollo abiertos, con un *load average* de entre 5 y 9 durante la medición), mientras que la máquina B estaba dedicada exclusivamente a la ejecución del benchmark. Eso sesga a la baja el speedup medido en la máquina A y, por tanto, exagera la ventaja de la B.

El efecto se ve en la propia consistencia interna de los datos: en la máquina B el costo por muestra del dartboard es de 0.404 ns en $N = 10^8$ y de 0.413 ns en $N = 10^9$, una variación del 2 %. En la máquina A los mismos valores son 0.595 ns y 1.068 ns, una variación del 79 %. Un algoritmo de costo constante por muestra no debería presentar esa dispersión: la diferencia procede del ruido del sistema, no del algoritmo.

Por eso la comparación entre máquinas debe leerse con esa reserva, y por eso el protocolo de la sección 5 insiste en medir con el equipo en reposo. Reejecutar la máquina A sin procesos de fondo es la forma de cerrar el dato.

## 7.10. Qué se mantiene igual entre máquinas

Dos conclusiones del documento son independientes del hardware y no cambian con la máquina del benchmark:

- **El orden de los algoritmos.** El dartboard es más rápido y más preciso que Buffon porque su modelo no requiere trigonometría. Eso es una propiedad del algoritmo, no del procesador.
- **El efecto del aliasing.** La variante `omp_alias` es más lenta que la serial por una razón de análisis del compilador, idéntica en cualquier máquina que use `gcc` con el mismo nivel de optimización.

Lo que sí depende del hardware es la *forma* de la curva de escalamiento: hasta qué cantidad de hilos rinde el paralelismo, y dónde aparece la saturación.

# 8. Conclusiones

## 8.1. Hallazgos

1. **El generador de números aleatorios es el cuello de botella, no la aritmética**. GProf atribuye más de la mitad del tiempo muestreado a `rng_double` en ambos algoritmos. Esto invierte la intuición de las aplicaciones de álgebra densa y explica por qué el paralelismo rinde bien: el trabajo por muestra es independiente, solo el generador es compartido.

2. **El dartboard domina a Buffon en las dos dimensiones**. Es 5.5 veces más rápido por muestra (2.95 ns contra 16.30 ns, con 65 instrucciones contra 136) porque no necesita evaluar $\sin\theta$, y además es un estimador más preciso: su desviación típica es $1{,}642/\sqrt{N}$ contra $2{,}374/\sqrt{N}$, es decir, Buffon necesita unas 2.09 veces más muestras para la misma exactitud.

3. **El perfilado de memoria explica el cambio de comportamiento respecto del caso de estudio 2**. Con una tasa de fallo de caché L1 del 0.0 % frente al 22.4 % de la multiplicación de matrices, el conjunto de trabajo cabe en la L1 y los hilos no compiten por la caché. Por eso aquí el paralelismo escala mucho mejor: 3.85× con Buffon en un equipo de 12 hilos lógicos.

4. **El trabajo más barato por muestra escala peor**. El dartboard, con 2.95 ns por muestra, consigue solo 1.40× con dos hilos, mientras Buffon consigue 1.61×. Los costos fijos del paralelismo pesan proporcionalmente más cuando el trabajo es más barato.

5. **Un paralelismo correcto en su lógica puede ser nulo en la práctica**. La variante que accede al estado del generador a través de un puntero compartido resulta ser más lenta que la versión serial (0.58–0.73×), porque el compilador no puede demostrar que el acumulador no se solapa con ese estado y lo mantiene en memoria en cada iteración. Copiar el estado a una variable local del hilo recupera un factor de 3.8× en el dartboard y 2.6× en Buffon.

6. **La arquitectura de la CPU determina cuánto rinde el paralelismo**. En el benchmark entre el Intel i5-12450HX (arquitectura híbrida de 6 núcleos de alto rendimiento más 2 de bajo rendimiento) y el AMD Ryzen 7 7735HS (8 núcleos Zen 3 homogéneos), la máquina B alcanza speedups de 6.89× y 9.56× frente a los 2.80× y 3.67× de la A, y su rendimiento es entre un 47 % y un 253 % mayor. Al paralelizar, los hilos de una CPU heterogénea acaban ejecutándose también sobre los núcleos lentos, lo que degrada el promedio del equipo.

7. **La validación de efectividad de un método estocástico es por convergencia, no por igualdad**. Con 20 repeticiones por configuración, todas las medias están a menos de $0{,}4\sigma$ teóricas del valor real de $\pi$, y las dispersiones medidas coinciden con las previstas por la teoría del error.

## 8.2. Limitaciones

- Las mediciones de la máquina A se realizaron con el equipo en uso habitual, con procesos ajenos en ejecución (*load average* de 5 a 9), mientras que las de la máquina B se hicieron sobre un equipo dedicado. Eso subestima el speedup de la A y exagera la ventaja de la B. La evidencia interna es que el costo por muestra del dartboard varía un 2 % entre $N = 10^8$ y $N = 10^9$ en la máquina B, y un 79 % en la A, cuando el costo por muestra de un algoritmo de este tipo no debería depender de $N$. Repetir la medición de la A con el equipo en reposo es el ajuste pendiente.

- GProf es un perfilador por muestreo y su cobertura es parcial: con la instrumentación de `-pg` el tiempo real de la corrida fue de 21.9 s para Buffon con $N = 10^9$, de los cuales solo 3.44 s quedaron atribuidos a muestras, porque el overhead de la propia instrumentación no se muestrea. Los porcentajes de GProf son por tanto proporciones del tiempo muestreado, y las cifras absolutas de costo por muestra del documento provienen de mediciones directas con `clock_gettime`.

- Se implementaron cuatro variantes de paralelización de un conjunto mayor de opciones posibles. Por ejemplo, no se exploraron las variantes con partición dinámica del trabajo, ni la vectorización manual del predicado del dartboard con instrucciones SIMD, que exigiría desacoplar la generación de números aleatorios del test.

- El perfilado con Cachegrind usa $N = 10^7$ en lugar de $10^9$, por el costo de la simulación. La tasa de fallo es una propiedad del patrón de acceso y no debería variar con $N$, pero no se verificó para los tamaños mayores. Tampoco se profileron los algoritmos en la máquina B, de modo que no se puede comparar la tasa de fallo entre las dos cachés.

## 8.3. Trabajo futuro

- Completar el benchmark entre las dos máquinas y extenderlo a más equipos para caracterizar el comportamiento del paralelismo en función de la cantidad de núcleos físicos y de la jerarquía de caché.

- Combinar la reducción privada con un *schedule* balanceado dinámicamente, que es lo que la literatura de OpenMP muestra como óptimo cuando el trabajo por iteración es desigual.

- Vectorizar el predicado del dartboard: si se genera un bloque de números aleatorios con un generador SIMD-friendly y se acumulan cuatro resultados booleanos a la vez, el predicado podría evaluarse con instrucciones vectoriales. Es la vía para cerrar la diferencia de costo con Buffon.

- Evaluar el estimador de Buffon con la extensión de Laplace (dos rejillas de líneas perpendiculares), que reduce el número de lanzamientos necesarios para una precisión dada.

# 9. Referencias


- **Buffon, G.-L. (1777)** — *Essai sur la théorie de la probabilité*. Formulación original del problema de la aguja. Referencia secundaria en `Buffon's needle problem`, Wikipedia.
- **Jörg Arndt y Christoph Haenel** — *Pi – Unleashed: Exploring the infinite*, Springer. Capitulado 1, pág. 39: método del dartboard para la estimación de $\pi$, con implementación de referencia en C. Disponible en `books.google.com`.
- **Blackman, D. y Vigna, S.** — *Scrambled Linear Pseudorandom Number Generators*, ACM Transactions on Mathematical Software, 47(4), 2021. Referencia del generador `xoshiro256**` y del mezclador `SplitMix64` empleados.
- **OpenMP Application Programming Interface Examples, Version 6.0.1 (noviembre 2025)** — ejemplos oficiales de la API OpenMP. Secciones 3.1 (`ploop.1.c`), 3.8 (`collapse.1.c`) y 5.3.5 (`task_dep.5.c`) utilizados en el análisis de opciones de paralelización.
- **OpenMP Application Programming Interface Specification, Version 6.0 (noviembre 2024)** — definición de las directivas, cláusulas `reduction`, `schedule` y `num_threads`, y de la semántica de las variables compartidas y privadas.
- **GNU `gprof`** — perfilador por muestreo con instrumentación en tiempo de compilación (`-pg`). Manual en `sourceware.org/binutils/docs`.
- **Valgrind:** `cachegrind` — simulador de caché del proyecto Valgrind. Documentación en `valgrind.org/docs/manual/cg-manual.html`.
- **POSIX.1-2017 (IEEE Std 1003.1-2017)** — `clock_gettime`, `getrusage`, `rand_r`, `drand48`. Especificación de la API POSIX utilizada para la medición del tiempo y de la memoria residente.
- **Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2** — modelo de caché jerárquica, líneas de caché y concepto de *false sharing*.
- **Brendan Gregg** — ***Systems Performance*** y su blog `brendangregg.com` — referencias sobre las herramientas de línea de comandos para rendimiento en Linux y sobre el uso del mínimo de las repeticiones como estimador robusto frente al ruido del sistema.
- **Material del curso de Computación de Alto Rendimiento** — presentaciones y grabaciones de clase (OpenMP, benchmark, profiling) utilizadas como referencia conceptual durante el desarrollo del reto.